QCC Rendering Foundation - Von Grund auf aufbauen

🎯 Strategischer Startpunkt: Interfaces & Contracts

Warum zuerst Interfaces?

- **Klare Contracts** definieren bevor wir implementieren
- **Testbarkeit** von Anfang an sicherstellen
- **Dependency Injection** ermöglichen
- **Einheitliche APIs** für alle Components

📋 Phase 3 Start-Reihenfolge

Schritt 1: Interfaces definieren (Tag 1)

```
📁 includes/infrastructure/interfaces/
├── interface-qcc-renderable.php       # Basis für alle Rendering-Components
├── interface-qcc-atom.php             # Atom-spezifisches Interface
├── interface-qcc-molecule.php         # Molecule-spezifisches Interface
├── interface-qcc-builder.php          # Builder-spezifisches Interface
└── interface-qcc-templatable.php      # Template-Support Interface
```

Schritt 2: Base-Classes erstellen (Tag 2)

```
📁 includes/rendering/base/
├── abstract-qcc-base-atom.php         # Abstract Base für alle Atoms
├── abstract-qcc-base-molecule.php     # Abstract Base für alle Molecules
├── abstract-qcc-base-builder.php      # Abstract Base für alle Builder
└── abstract-qcc-base-component.php    # Gemeinsame Basis für alle
```

Schritt 3: Registry & Factory (Tag 3)

```
📁 includes/rendering/core/
├── class-qcc-component-registry.php   # Component Registration & Management
├── class-qcc-component-factory.php    # Component Creation & Injection
└── class-qcc-template-manager.php     # Template Loading & Caching
```

Schritt 4: Erste Atoms implementieren (Tag 4-5)

```
📁 includes/rendering/atoms/inputs/
├── class-qcc-percentage-input.php      # Prozent-Eingabe
├── class-qcc-currency-input.php        # Währungs-Eingabe
└── class-qcc-select-input.php          # Dropdown-Auswahl
```

💻 Konkrete Implementation - Schritt 1: Interfaces

1. Basis-Interface: QCC_Renderable

```php
<?php
/**
 * Base Interface für alle Rendering-Komponenten
 */
interface QCC_Renderable {
    /**
     * Rendert die Komponente als HTML
     * @param array $data Component-spezifische Daten
     * @param array $attributes HTML-Attribute
     * @return string Generated HTML
     */
    public function render($data = array(), $attributes = array());

    /**
     * Validiert Input-Daten
     * @param array $data Zu validierende Daten
     * @return bool|WP_Error Validation Result
     */
    public function validate_data($data);

    /**
     * Gibt benötigte Assets zurück
     * @return array Array mit CSS/JS Dependencies
     */
    public function get_required_assets();

    /**
     * Erstellt Cache-Key für Component
     * @param array $data Component-Daten
     * @return string Unique Cache Key
     */
    public function get_cache_key($data);
}
```

2. Atom-Interface: QCC_Atom

```php
<?php
/**
 * Interface für Atomic Components (kleinste UI-Bausteine)
 */
interface QCC_Atom extends QCC_Renderable {
    /**
     * Standard-Attribute für Atom
     * @return array Default HTML Attributes
     */
    public function get_default_attributes();

    /**
     * Sanitisiert Input-Werte
     * @param mixed $value Input-Wert
     * @return mixed Sanitized Value
     */
    public function sanitize_input($value);

    /**
     * Template-Name für Atom
     * @return string Template Filename
     */
    public function get_template_name();

    /**
     * CSS-Klassen für Atom
     * @param array $data Component-Daten
     * @return string CSS Classes
     */
    public function get_css_classes($data);
}
```

3. Molecule-Interface: QCC_Molecule

```php
```

```php
<?php
/**
 * Interface für Molecule Components (Kombination von Atoms)
 */
interface QCC_Molecule extends QCC_Renderable {
    /**
     * Child-Components definieren
     * @return array Array von Child-Component-Names
     */
    public function get_child_components();

    /**
     * Components zusammenbauen
     * @param array $components Gerenderte Child-Components
     * @return string Assembled HTML
     */
    public function assemble_components($components);

    /**
     * Layout-Template für Molecule
     * @return string Layout Template Name
     */
    public function get_layout_template();

    /**
     * Data-Mapping für Child-Components
     * @param array $data Input-Daten
     * @return array Mapped Data für Children
     */
    public function map_data_to_children($data);
}
```

4. Builder-Interface: QCC_Builder

```php
```

```php
<?php
/**
 * Interface für Builder Components (Complex UI Sections)
 */
interface QCC_Builder extends QCC_Renderable {
    /**
     * Build-Prozess definieren
     * @param array $config Build-Konfiguration
     * @return string Complete Section HTML
     */
    public function build($config);

    /**
     * Sub-Sections definieren
     * @return array Array von Sub-Section-Definitions
     */
    public function get_sub_sections();

    /**
     * Event-Hooks für Builder
     * @return array Array von Hook-Definitions
     */
    public function get_hooks();

    /**
     * Dependencies für Builder
     * @return array Required Components/Services
     */
    public function get_dependencies();
}
```

🏗️ Schritt 2: Base-Classes implementieren

1. Abstract Base Component

```php
php
```

```php
<?php
/**
 * Abstract Base für alle Rendering-Components
 */
abstract class QCC_Base_Component {
    protected $translator;
    protected $template_manager;
    protected $asset_manager;
    protected $cache_manager;

    public function __construct() {
        $this->translator = QCC_Service_Container::get('translator');
        $this->template_manager = QCC_Service_Container::get('template_manager');
        $this->asset_manager = QCC_Service_Container::get('asset_manager');
        $this->cache_manager = QCC_Service_Container::get('cache_manager');
    }

    /**
     * Standard Cache-Key Generation
     */
    public function get_cache_key($data) {
        return sprintf(
            'qcc_%s_%s_%s',
            get_class($this),
            md5(serialize($data)),
            get_locale()
        );
    }

    /**
     * Standard Asset-Loading
     */
    public function get_required_assets() {
        return array(
            'css' => array(),
            'js' => array(),
            'dependencies' => array()
        );
    }

    /**
     * Standard Data-Validation
     */
    public function validate_data($data) {
        if (!is_array($data)) {
            return new WP_Error('invalid_data', 'Data must be array');
```

```
        }

        $required_fields = $this->get_required_fields();
        foreach ($required_fields as $field) {
            if (!isset($data[$field])) {
                return new WP_Error('missing_field', sprintf('Required field %s missing', $field));
            }
        }

        return true;
    }

    /**
     * Override in child classes
     */
    abstract protected function get_required_fields();
}
```

2. Abstract Base Atom

php

```php
<?php
/**
 * Abstract Base für alle Atom-Components
 */
abstract class QCC_Base_Atom extends QCC_Base_Component implements QCC_Atom {
    protected $default_attributes = array();
    protected $css_base_class = '';
    protected $template_name = '';

    /**
     * Standard Atom Rendering
     */
    public function render($data = array(), $attributes = array()) {
        // Validation
        $validation = $this->validate_data($data);
        if (is_wp_error($validation)) {
            return $this->render_error($validation);
        }

        // Caching prüfen
        $cache_key = $this->get_cache_key($data);
        $cached = $this->cache_manager->get($cache_key);
        if ($cached !== false) {
            return $cached;
        }

        // Template-Daten vorbereiten
        $template_data = $this->prepare_template_data($data, $attributes);

        // Template rendern
        $html = $this->template_manager->render($this->get_template_name(), $template_data);

        // Cache speichern
        $this->cache_manager->set($cache_key, $html, 3600); // 1 Stunde

        return $html;
    }

    public function get_default_attributes() {
        return $this->default_attributes;
    }

    public function get_template_name() {
        return $this->template_name ?: $this->generate_template_name();
    }
```

```php
public function get_css_classes($data) {
    $classes = array($this->css_base_class);

    // Status-spezifische Klassen
    if (isset($data['error']) && $data['error']) {
        $classes[] = 'qcc-error';
    }
    if (isset($data['required']) && $data['required']) {
        $classes[] = 'qcc-required';
    }

    return implode(' ', array_filter($classes));
}

/**
 * Standard Input-Sanitization
 */
public function sanitize_input($value) {
    if (is_string($value)) {
        return sanitize_text_field($value);
    }
    if (is_numeric($value)) {
        return floatval($value);
    }
    return $value;
}

/**
 * Template-Daten für Rendering vorbereiten
 */
protected function prepare_template_data($data, $attributes) {
    return array(
        'data' => $data,
        'attributes' => array_merge($this->get_default_attributes(), $attributes),
        'css_classes' => $this->get_css_classes($data),
        'id' => $this->generate_id($data),
        'translator' => $this->translator
    );
}

/**
 * Unique ID für Component generieren
 */
protected function generate_id($data) {
    return isset($data['id']) ? $data['id'] : uniqid('qcc_');
}
```

```php
    /**
     * Template-Name automatisch generieren
     */
    protected function generate_template_name() {
        $class_name = get_class($this);
        return strtolower(str_replace(array('QCC_', '_'), array('', '-'), $class_name)) . '.php';
    }

    /**
     * Error-Rendering
     */
    protected function render_error($error) {
        return sprintf(
            '<div class="qcc-component-error">%s</div>',
            esc_html($error->get_error_message())
        );
    }
}
```

🚀 Schritt 3: Component Registry

Component Registry für Dependency Injection

```php
```

```php
<?php
/**
 * Component Registry für Registration und Management
 */
class QCC_Component_Registry {
    private static $instance = null;
    private $atoms = array();
    private $molecules = array();
    private $builders = array();
    private $instances = array();

    public static function get_instance() {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    /**
     * Atom registrieren
     */
    public function register_atom($name, $class_name) {
        $this->atoms[$name] = array(
            'class' => $class_name,
            'dependencies' => $this->analyze_dependencies($class_name)
        );
    }

    /**
     * Molecule registrieren
     */
    public function register_molecule($name, $class_name) {
        $this->molecules[$name] = array(
            'class' => $class_name,
            'dependencies' => $this->analyze_dependencies($class_name)
        );
    }

    /**
     * Builder registrieren
     */
    public function register_builder($name, $class_name) {
        $this->builders[$name] = array(
            'class' => $class_name,
            'dependencies' => $this->analyze_dependencies($class_name)
        );
```

```php
}

/**
 * Component-Instanz abrufen (mit Lazy Loading)
 */
public function get($name, $type = 'auto') {
    $component_key = $type . '_' . $name;

    // Bereits instanziiert?
    if (isset($this->instances[$component_key])) {
        return $this->instances[$component_key];
    }

    // Component-Definition finden
    $definition = $this->find_component_definition($name, $type);
    if (!$definition) {
        throw new Exception("Component {$name} not found");
    }

    // Dependencies laden
    $dependencies = $this->load_dependencies($definition['dependencies']);

    // Instanz erstellen
    $class_name = $definition['class'];
    $instance = new $class_name();

    // Dependencies injizieren (falls Constructor das unterstützt)
    if (method_exists($instance, 'set_dependencies')) {
        $instance->set_dependencies($dependencies);
    }

    // Cache für weitere Verwendung
    $this->instances[$component_key] = $instance;

    return $instance;
}

/**
 * Alle Components eines Typs abrufen
 */
public function get_all_atoms() {
    return array_keys($this->atoms);
}

public function get_all_molecules() {
    return array_keys($this->molecules);
}
```

```php
    public function get_all_builders() {
        return array_keys($this->builders);
    }

    /**
     * Component-Definition finden
     */
    private function find_component_definition($name, $type) {
        if ($type === 'atom' && isset($this->atoms[$name])) {
            return $this->atoms[$name];
        }
        if ($type === 'molecule' && isset($this->molecules[$name])) {
            return $this->molecules[$name];
        }
        if ($type === 'builder' && isset($this->builders[$name])) {
            return $this->builders[$name];
        }

        // Auto-Detection
        if (isset($this->atoms[$name])) return $this->atoms[$name];
        if (isset($this->molecules[$name])) return $this->molecules[$name];
        if (isset($this->builders[$name])) return $this->builders[$name];

        return null;
    }

    /**
     * Dependencies analysieren (vereinfacht)
     */
    private function analyze_dependencies($class_name) {
        // TODO: Reflection-basierte Dependency-Analyse
        return array();
    }

    /**
     * Dependencies laden
     */
    private function load_dependencies($dependencies) {
        $loaded = array();
        foreach ($dependencies as $dependency) {
            $loaded[$dependency] = QCC_Service_Container::get($dependency);
        }
        return $loaded;
    }
}
```

📋 Nächste Schritte nach Foundation

Tag 1: Interfaces erstellen ✅

- Alle 5 Interface-Dateien implementieren
- Contracts für Atoms, Molecules, Builders definieren

Tag 2: Base-Classes erstellen ✅

- Abstract Base-Classes für wiederverwendbare Logik
- Standard-Implementierungen für Caching, Validation, etc.

Tag 3: Registry & Factory ✅

- Component-Registry für Dependency Injection
- Factory-Pattern für Component-Creation
- Template-Manager für HTML-Templates

Tag 4-5: Erste Atoms implementieren

- Percentage-Input, Currency-Input, Select-Input
- Mit konkreten Templates und Assets
- Unit-Tests für alle Atoms

Nach diesen 5 Tagen haben Sie eine solide Foundation für alle weiteren Components!

🎯 Warum diese Reihenfolge optimal ist

1. Interfaces zuerst = Klare Contracts

- **Testbarkeit** von Anfang an
- **Einheitliche APIs** für alle Components
- **Dependency Injection** möglich

2. Base-Classes = Code-Wiederverwendung

- **80% gemeinsame Logik** in Base-Classes
- **Konsistente Implementierung** aller Components
- **Weniger Bugs** durch getestete Base-Funktionalität

3. Registry = Lose Kopplung

- **Dependency Injection** für bessere Testbarkeit
- **Lazy Loading** für Performance

- **Einfache Component-Erweiterung**

Mit dieser Foundation können Sie dann systematisch Atoms → Molecules → Builders implementieren!

Sollen wir mit **Schritt 1 (Interfaces)** beginnen?